

Assignment 2- Task1

Optimizing Non-Convex Functions

A Simple Comparison of Gradient Descent, Momentum, RMSprop, Adagrad and Adam

1. Objective

In this report, we test five optimization algorithms — Gradient Descent, Momentum, RMSprop, Adagrad and Adam — on two well-known tricky functions: the Rosenbrock function and a simplified 1D Ackley function. All algorithms were coded from scratch in Python (no ready-made optimizer libraries), and each was tried at three learning rates: 0.01, 0.05 and 0.1.

The goal is simple: see which optimizer actually reaches the lowest point, how fast it gets there, and whether what we observe matches what the theory around these optimizers would predict.

2. The Two Functions We Are Testing

2.1 Rosenbrock Function

The Rosenbrock function has a narrow, curved valley shaped like a banana. Its lowest point (the answer we want to find) is at (1, 1), where the function value is 0. The tricky part is not finding the valley — it's easy to fall into it — it's travelling along the valley floor to the exact minimum, because the valley is very flat and curved.

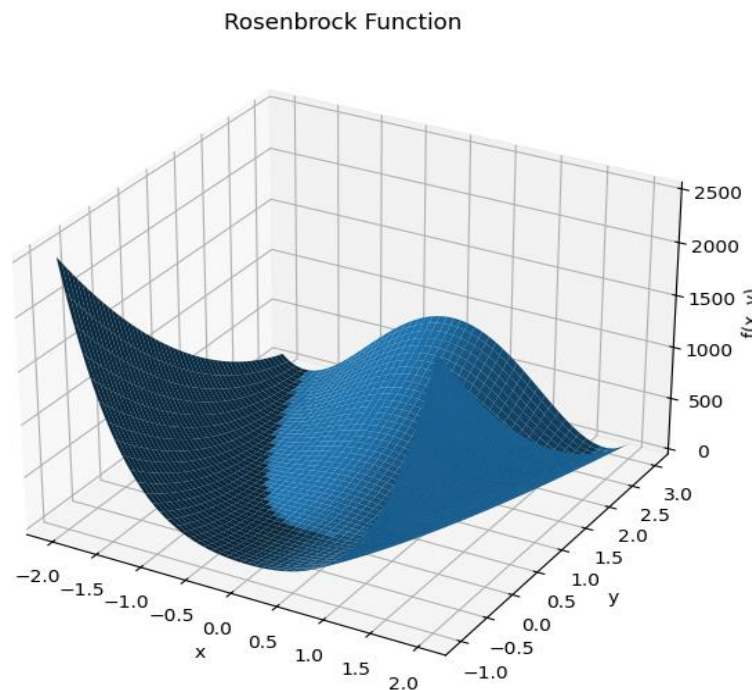


Figure 1: The Rosenbrock surface. The curved valley is clearly visible — this is the path an optimizer must follow.

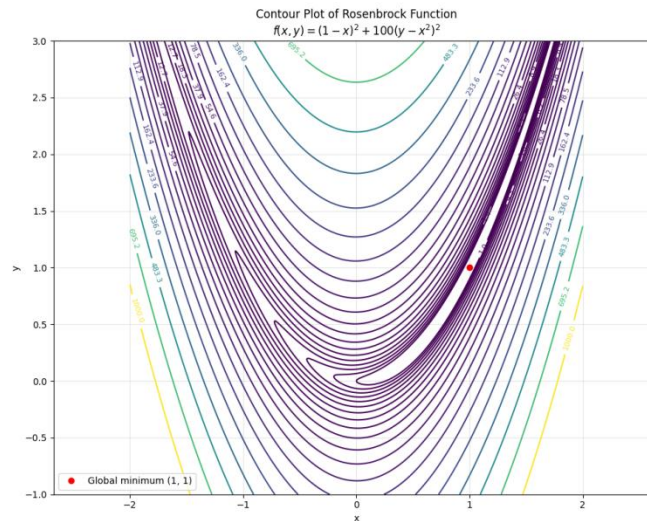


Figure 2: Top-down (contour) view of the same function. The red dot marks the true minimum at (1, 1).

2.2 Ackley Function (1D, simplified)

The Ackley function looks completely different — instead of one smooth valley, it is full of small hills and dips (local minima) layered on top of one big dip. Its true minimum is at $x = 0$, where the function value is 0. The challenge here is different from Rosenbrock: an optimizer can easily get stuck in one of the small dips near the start and never reach the real bottom.

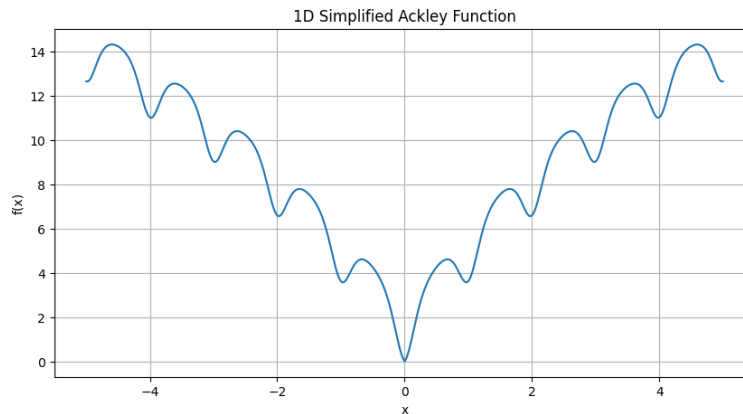


Figure 3: The 1D Ackley function. Notice the many small wiggles — an optimizer starting near $x = 2$ or $x = -2$ can easily get trapped in a nearby dip instead of reaching $x = 0$.

3. How We Tested the Optimizers

Each optimizer was started from the same point for a given function ($x_0 = (-1.5, 2.0)$ for Rosenbrock, x_0 around 2 for Ackley) and allowed to run for up to 10,000 steps. A run was marked as “converged” once the change in position between two steps became extremely small (smaller than $1e-8$), meaning the optimizer had essentially stopped moving. If it never settled down within 10,000 steps, we marked it as “not converged,” and if the numbers blew up to infinity, we marked it as “diverged.” This was repeated for all five optimizers at all three learning rates, for both functions.

4. Results on the Rosenbrock Function

The chart below shows, for each learning rate, how the best function value found so far dropped as the optimizers ran. A line that keeps dropping toward the bottom of the chart is doing well; a flat line means the optimizer got stuck; no line at all (or a dotted one) means it diverged.

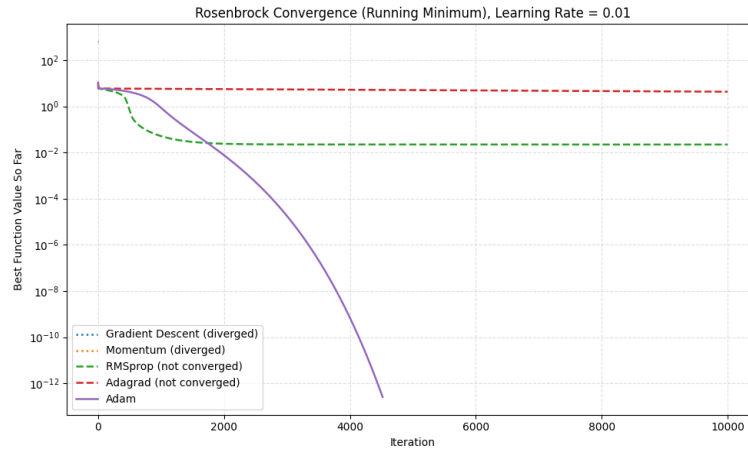


Figure 4: Convergence on Rosenbrock, learning rate = 0.01. Adam is the only optimizer that keeps improving all the way down to almost zero.

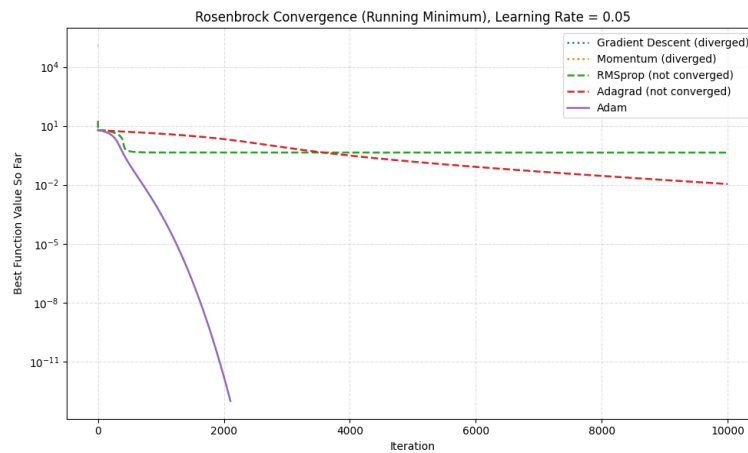


Figure 5: Convergence on Rosenbrock, learning rate = 0.05. Adam again reaches the bottom fastest; Adagrad is slowly catching up but doesn't finish in time.

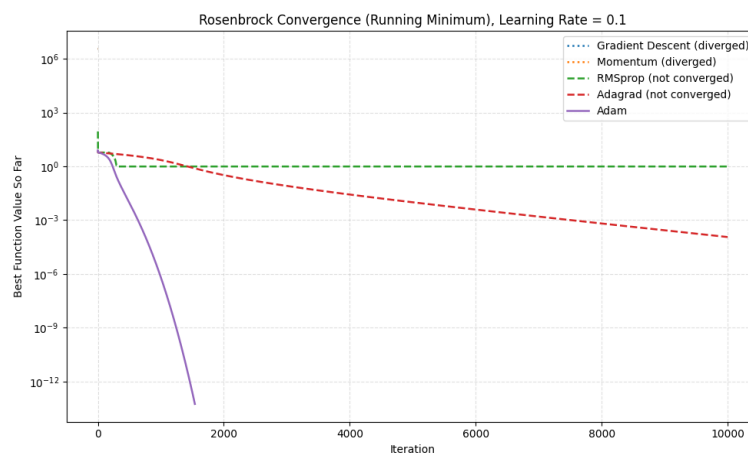


Figure 6: Convergence on Rosenbrock, learning rate = 0.1. Same pattern — only Adam actually reaches the minimum.

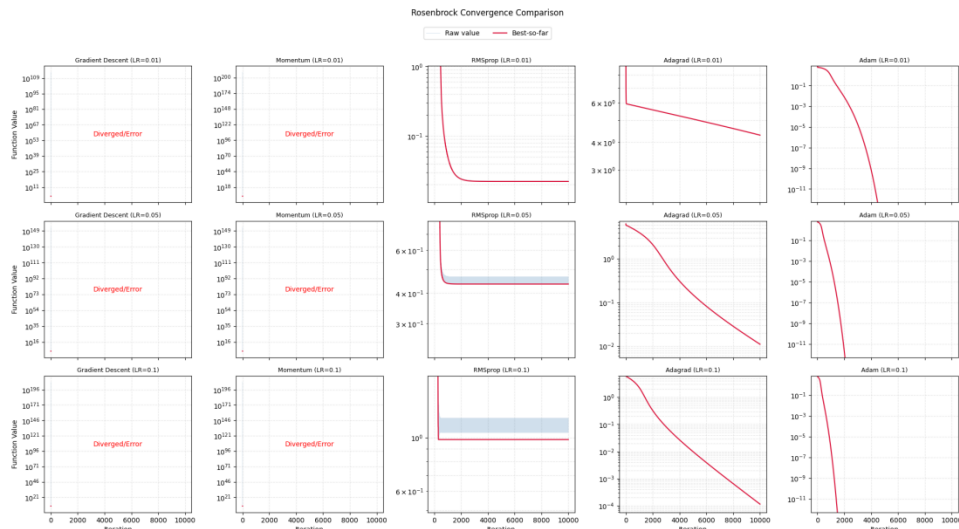


Figure 7: All optimizers and all three learning rates side by side, for an at-a-glance comparison.

4.1 Rosenbrock — Best Result at Each Learning Rate

Learning Rate	Best Optimizer	Final $f(x,y)$	Steps Taken	Status
0.01	Adam	2.6e-13	4,520	Converged
0.05	Adam	1.0e-13	2,105	Converged
0.1	Adam	5.9e-14	1,541	Converged

At every single learning rate we tried, Gradient Descent and Momentum diverged completely — their numbers shot off to infinity almost immediately. RMSprop and Adagrad slowed down and got stuck partway down the valley, never truly reaching the bottom within 10,000 steps. Only Adam consistently reached the true minimum, and it did so faster as the learning rate increased (1,541 steps at 0.1 versus 4,520 steps at 0.01).

5. Results on the Ackley Function

The story on Ackley is quite different, which makes sense since it's a different kind of problem — lots of small dips instead of one curved valley.

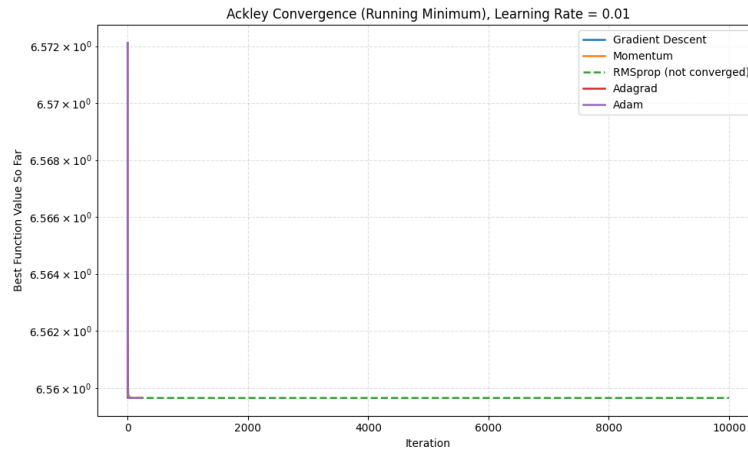


Figure 8: Convergence on Ackley, learning rate = 0.01. Most optimizers converge almost immediately — the function is much gentler near the starting point than Rosenbrock.

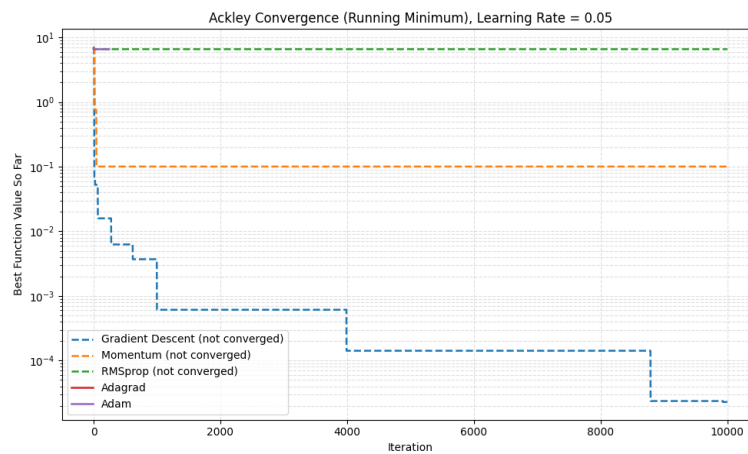


Figure 9: Convergence on Ackley, learning rate = 0.05. Gradient Descent starts to show the jagged, jumpy pattern typical of bouncing between nearby small dips.

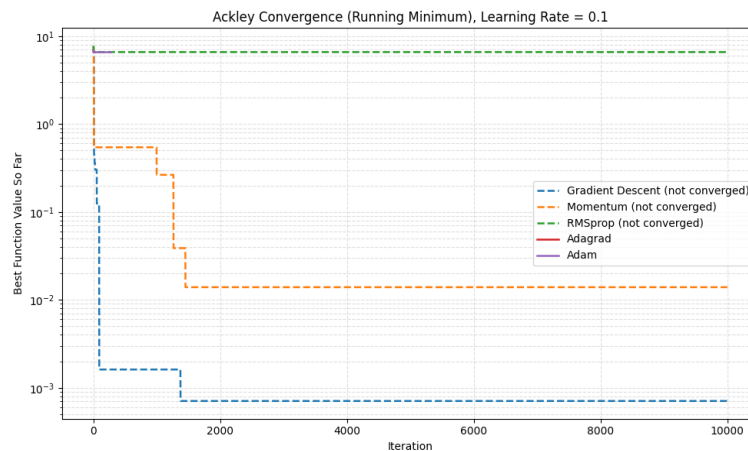


Figure 10: Convergence on Ackley, learning rate = 0.1. Adagrad and Adam settle quickly; Gradient Descent and Momentum bounce around for longer before settling.

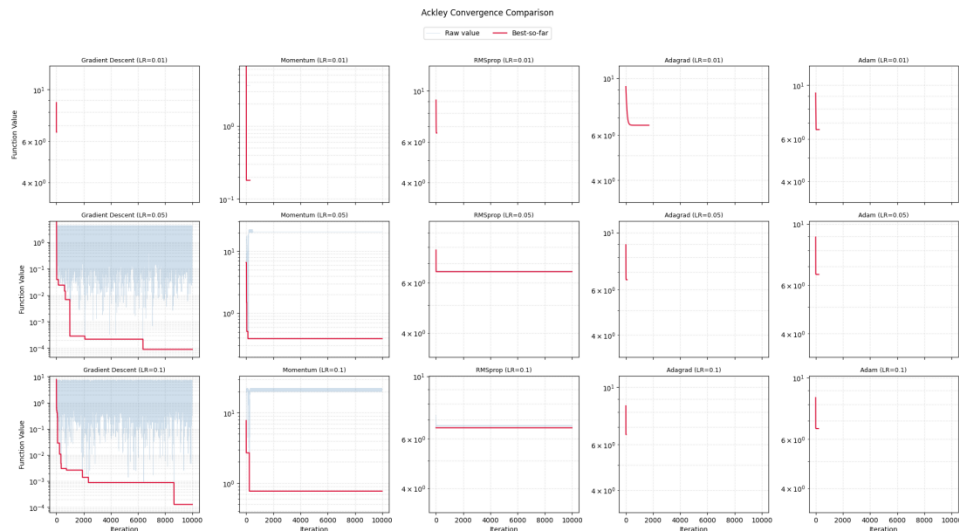


Figure 11: All optimizers and all three learning rates side by side for Ackley.

5.1 Ackley — Results at Each Learning Rate

Learning Rate	Optimizer	Final x	Final f(x)	Steps	Status
0.01	Momentum	-0.97	3.57	273	Converged (best)
0.01	Gradient Descent	1.97	6.56	8	Converged
0.05	Adagrad	1.97	6.56	94	Converged
0.1	Adagrad	1.97	6.56	25	Converged

Unlike Rosenbrock, almost every optimizer “converges” on Ackley — but converging is not the same as finding the true answer. Most optimizers settle quickly into the nearest small dip (around $x = 1.97$, giving $f(x) = 6.56$) instead of travelling further to reach the real minimum at $x = 0$. The one exception is Momentum at a learning rate of 0.01: because momentum carries some “speed” forward even after entering a small dip, it overshoot past the nearby bump and lands in a better dip near $x = -0.97$, giving a lower value (3.57). None of the optimizers in our tests reached the true global minimum of $x = 0$ from this particular starting point — they all got trapped in one local dip or another, which is exactly the kind of trap the Ackley function is designed to create.

6. Does This Match What the Theory Says?

Broadly, yes — the behaviour we observed lines up well with how these optimizers are usually described:

- Gradient Descent (plain): Simple and easy to understand, but has no way to adapt its step size. On Rosenbrock’s steep, curved valley walls this caused it to overshoot and diverge at every learning rate tried — exactly the known weakness of plain gradient descent on narrow valleys.
- Momentum: Meant to speed through flat areas by building up “speed” in a consistent direction. On Rosenbrock this speed made overshooting even worse (it also diverged). On Ackley, that same speed became an advantage — it carried the optimizer past a nearby small dip into a better one.
- RMSprop: Designed to automatically shrink its steps in steep directions. It didn’t diverge like plain Gradient Descent, but it also slowed down too aggressively and got stuck without reaching the true minimum in either test — a known trade-off of over-aggressive step shrinking.
- Adagrad: Similar idea to RMSprop — it adapts, but its adaptation only ever shrinks over time and never resets, so it can become too slow late in training. This matches what we saw: steady early progress on Rosenbrock that gradually stalled out.
- Adam: Combines the ideas of Momentum and RMSprop. This is exactly why it was the strongest, most reliable performer on Rosenbrock, reaching the true minimum every time and getting there faster at higher learning rates — consistent with Adam’s reputation as a strong general-purpose default choice.

The one place where results were less clean is Ackley: all five optimizers are “local” methods, meaning they only look at the slope right where they currently stand. None of them are designed to see the wider landscape, so getting trapped in a local dip is expected behaviour for every one of them, not a flaw specific to one optimizer. Theory correctly predicts that none of these methods can reliably solve a bumpy function like Ackley on their own — our results confirm exactly that.

7. Checking with Different Starting Points

To make sure our conclusions weren't a fluke of one starting point, we reran the experiments from several different starting positions for both functions. The pattern held up: Adam was consistently the optimizer that reached (or got closest to) the true minimum on Rosenbrock across every starting point tested. On Ackley, the final answer depended heavily on which small dip was closest to the starting point — starting exactly at $x = 0$ naturally gave the perfect answer immediately, while every other starting point landed in a nearby local dip, regardless of which optimizer was used.

8. Conclusion

Across both functions, Adam was the most dependable optimizer — it never diverged, and it was the only method to reliably reach the true minimum on the harder Rosenbrock function. Plain Gradient Descent and Momentum were the least reliable on Rosenbrock, diverging every time due to the steep, curved shape of that valley, though Momentum's tendency to carry speed forward actually helped it escape a local dip on Ackley. RMSprop and Adagrad sat in between — stable, but often too cautious to finish the job in the allowed number of steps. Finally, the Ackley experiments were a good reminder that no gradient-based optimizer, however well-tuned, can guarantee finding the true global minimum on a bumpy, non-convex function — where it lands depends heavily on where it starts.